

ASSESSMENT TOOL-CO2-AT3

Debugging and Optimization Task

Name: S. Bhuvanesh

Reg No: 192521004

Q5. Debugging Binary Search Code

The following Binary Search implementation fails for certain inputs due to incorrect boundary updates. Identify all logical errors and explain their causes. Debug the code to handle all edge cases correctly. Optimize the implementation to avoid overflow in mid calculation. Analyze its performance in best, average, and worst cases. Rewrite the corrected code with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low < high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            high = mid - 1
        else:
            low = mid + 1
    return -1
```

Introduction:

Binary Search is one of the fastest searching algorithms used to locate an element in a **sorted array**. Unlike Linear Search, which checks every element one by one, Binary Search repeatedly divides the search space into two halves. In every iteration, it compares the search key with the middle element of the array. Depending on the comparison result, one half of the array is discarded, thereby reducing the search space significantly. Because Binary Search reduces the search area by half during each iteration, its time complexity is **$O(\log n)$** , making it much faster than Linear Search for large datasets.

However, the given Binary Search program contains several logical mistakes. These mistakes prevent the algorithm from correctly searching all elements and may even search in the wrong direction. The following sections identify each error, explain why it occurs, and show the corrected implementation.

Objective:

- To identify all the logical errors present in the given Binary Search program.
- To understand why the program fails for certain inputs.
- To correct the loop condition so that every possible element is checked.
- To correct the boundary updates (low and high) so the search proceeds in the correct direction.
- To optimize the middle index calculation to avoid integer overflow and improve code reliability.
- To implement a fully corrected Binary Search algorithm capable of handling all edge cases efficiently.

Given code:

```
1 def binary_search(arr, key):
2     low, high = 0, len(arr)-1
3
4     while low < high:
5
6         mid = (low + high) // 2
7
8         if arr[mid] == key:
9             return mid
10
11        elif arr[mid] < key:
12            high = mid - 1
13
14        else:
15            low = mid + 1
16
17    return -1
```

Error 1: Incorrect Loop Condition

Faulty Code

while low < high:

Why is it Wrong?

The Binary Search should continue searching until **low becomes greater than high**.

The condition

while low < high

stops the loop when

low == high

Therefore, the last remaining element is never checked.

Example

Array

[10,20,30,40,50]

Searching for

50

Iteration 1

low = 0

high = 4

mid = 2

arr[mid] = 30

Move right

low = 3

Iteration 2

low = 3

high = 4

mid = 3

arr[mid] = 40

Move right

low = 4

Now

low = 4

high = 4

Loop condition

$4 < 4$

False

The loop exits without checking

`arr[4]=50`

Hence, the element is incorrectly reported as **not found**.

Correct Code

`while low <= high:`

Code after Error 1 Correction

```
main.c
1  def binary_search(arr, key):
2
3      low = 0
4      high = len(arr)-1
5
6      while low <= high:
7
8          mid = (low + high)//2
9
10         if arr[mid] == key:
11             return mid
12
13         elif arr[mid] < key:
14             high = mid-1
15
16         else:
17             low = mid+1
18
19     return -1
```

Error 2: Wrong Boundary Update

Faulty Code

`elif arr[mid] < key:`

`high = mid - 1`

Why is it Wrong?

If

`arr[mid] < key`

the required element must be in the **right half** of the array.

But the program searches the **left half**, which is incorrect.

Example

Array

10 20 30 40 50

Search

40

Current values

`low = 0`

`high = 4`

`mid = 2`

`arr[mid]=30`

Since

`30 < 40`

Correct Search

Right Side

Correct update

`low = mid + 1`

Wrong update

`high = mid -1`

This removes the correct portion of the array.

Correct Code

`elif arr[mid] < key:`

low = mid +

Code after Error 2 Correction

```
main.c
1 def binary_search(arr,key):
2
3     low=0
4     high=len(arr)-1
5
6     while low<=high:
7
8         mid=(low+high)//2
9
10        if arr[mid]==key:
11            return mid
12
13        elif arr[mid]<key:
14            low=mid+1
15
16        else:
17            low=mid+1
18
19    return -1
```

Error 3: Wrong Else Boundary

Faulty Code

else:

low = mid + 1

Why is it Wrong?

If

arr[mid] > key

the search must continue in the **left half**.

Instead, the program again moves to the right side.

Example

Array

10 20 30 40 50

Searching

20

Current values

mid = 2

arr[mid]=30

Since

$30 > 20$

Correct search

Left Half

Correct update

high = mid -1

Wrong update

low = mid +1

Again the correct element is skipped.

Correct Code

else:

high = mid - 1

Code after Error 3 Correction

```

main.c
1  def binary_search(arr, key):
2
3      low=0
4      high=len(arr)-1
5
6      while low<=high:
7
8          mid=(low+high)//2
9
10         if arr[mid]==key:
11             return mid
12
13         elif arr[mid]<key:
14             low=mid+1
15
16         else:
17             high=mid-1
18
19     return -1
20

```

Error 4: Overflow in Mid Calculation

Faulty Code

`mid=(low+high)//2`

Why is it Wrong?

In languages such as

- C
- C++
- Java

adding

`low + high`

may exceed the integer limit and cause an overflow.

Although Python integers do not overflow in this way, using the optimized formula is considered best practice.

Correct Formula

`mid = low + (high-low)//2`

Final Correct Program

main.c

```
1 def binary_search(arr,key):
2
3     low=0
4     high=len(arr)-1
5
6     while low<=high:
7
8         mid=low+(high-low)//2
9
10        if arr[mid]==key:
11            return mid
12
13        elif arr[mid]<key:
14            low=mid+1
15
16        else:
17            high=mid-1
18
19    return -1
20
21 arr=[10,20,30,40,50,60,70]
22
23 print("Sorted Array =",arr)
24
25 key=int(input("Enter Search Element : "))
26
27 result=binary_search(arr,key)
28
29 if result!=-1:
30     print("Element Found at Index =",result)
31 else:
32     print("Element Not Found")
```

Algorithm

1. Start the program.
2. Read the sorted array and the search key from the user.
3. Initialize $low = 0$ and $high = n - 1$.
4. Repeat while $low \leq high$:
 - Calculate the middle index using $mid = low + (high - low) // 2$.
 - If the middle element equals the key, return its index.
 - If the middle element is smaller than the key, update $low = mid + 1$ and search the right half.
 - Otherwise, update $high = mid - 1$ and search the left half.
5. If the loop ends without finding the key, return -1.
6. Display whether the element was found or not.
7. Stop the program.

Pseudocode

BEGIN

Input Sorted Array

Input Search Key

$low \leftarrow 0$

$high \leftarrow \text{length}(\text{array}) - 1$

WHILE $low \leq high$

$mid \leftarrow low + (high - low) / 2$

 IF $\text{array}[mid] == \text{key}$

 RETURN mid

 ELSE IF $\text{array}[mid] < \text{key}$

$low \leftarrow mid + 1$

 ELSE

$high \leftarrow mid - 1$

END WHILE

RETURN -1

END